Buzz Fly — Git & GitHub Workflow

A practical guide for a 6-person team using a single shared repo. Read once, refer back when stuck.

Do you (Karim) need to `git clone`?

No. You already have the project on your computer — you created it. Cloning is for **downloading** a repo you don't have yet.

- **You (Karim):** skip the clone. Your repo is already linked to GitHub. Just `git pull` to get teammates' changes.
- **Your 5 teammates:** they each clone **once** to download a copy onto their own machines.

> **Mental model:** GitHub is like a shared Google Drive folder. You uploaded the project — you don't need to download what you uploaded. Your friends download it (clone) so they have their own copy to work on.

The big picture

```
    GitHub (cloud / shared)
                          ↑ git push                  ↓ git pull / git clone

    Your computer    →   git checkout -b   →   edit files   →   git add   →   git commit
```

Two halves to remember:

- **Local** (`add` , `commit` , `checkout` , `branch`) — happens on your computer. Free, instant, reversible.
- **Remote** (`push` , `pull` , `clone`) — talks to GitHub. Syncs you with the team.

Step-by-step: every command explained

1 First time only — for teammates

```
git clone https://github.com/Karimkimoz74/buzz-fly.git
```

What: Downloads the entire project (all files + full history) from GitHub to their computer.
Why: They need a local copy to edit. Internet folder → their hard drive.
When: Once per teammate, ever. After this they never clone again.

```
cd buzz-fly
```

What: Move into the folder that was just created.
Why: All other git commands must be run **inside** the project folder.

2 Every time you start a new task — everyone, including you

```
git checkout main
```

What: Switch to the `main` branch (the "official" version).
Why: Always start from the latest official code, not from someone's half-finished work.

```
git pull
```

What: Download any new changes teammates pushed since you last looked.
Why: If Rawan merged her hotel page yesterday, your local copy doesn't know. `pull` syncs you up.
Analogy: Refreshing the page to see new messages.

```
git checkout -b feature/my-task
```

What: Create a new branch called `feature/my-task` and switch to it.
Why: A branch is a copy of the project where you can experiment **without breaking the official version**.
Naming: Replace `my-task` with what you're doing — `feature/home-page`, `feature/login-form`, `fix/navbar-mobile`.

> **Why branches matter:** If everyone edited `main` directly, you'd all overwrite each other constantly. Branches = each person works in their own sandbox, then merges that sandbox into `main` when ready.

3 While you're working — save your progress

```
git status
```

What: Shows which files you've changed (not committed yet).
Why: Sanity check — "did I touch what I think I touched?"

```
git add index.html css/pages/home.css
```

What: Mark these specific files for your next save (commit).
Why: You might have changed 10 files but only want to save 3 right now.
Tip: Avoid `git add .` — saves everything blindly, risky if you have secrets or junk.

```
git commit -m "home: add hero section"
```

What: Saves a snapshot of the staged files to your local history with a label.
Why: A commit is a save point. You can always come back to it. The message tells future-you (and your team) what changed.
Important: This only saves on your computer. GitHub doesn't know about it yet.

> **Make commits small and frequent.** Don't write 3 hours of code then commit once. Commit every time you finish a small piece — "added the navbar", "styled the hero", "fixed mobile spacing". Easier to undo a small mistake than untangle a giant one.

4 Share your work with the team

```
git push -u origin feature/my-task
```

What: Upload your branch + all its commits to GitHub.
Why: Now your teammates can see and review it.

- `origin` = the GitHub repo
- `feature/my-task` = your branch name
- `-u` = "remember this connection so next time I just type `git push`"

After the first push, every following push is just:

```
git push
```

5 Get your work into `main` — Pull Request (PR)

A PR is not a git command — it's done on **github.com** in the browser.

1. After you push, go to your repo on github.com
2. GitHub shows a yellow banner: **"Compare & pull request"** — click it
3. **Title:** what you did (e.g. "Home page hero section")
4. **Description:** 2–3 bullets explaining what changed + a screenshot if it's UI
5. **Reviewers:** pick a teammate
6. They look at your code, leave comments, click **Approve**
7. Click the green **Merge pull request** button → use **Squash and merge**
8. Your branch is now part of `main`. Done.

Why PRs? Two reasons: code review (a teammate catches your bugs before they reach `main`) and discussion (comments live next to the actual code lines).

6 After your PR is merged — clean up

```
git checkout main
git pull
git branch -d feature/my-task
```

Switch back to `main`, pull the latest (which now includes **your** merged work), delete your finished branch. Old branches accumulate clutter — delete once they've served their purpose.

Then start the cycle again with `git checkout -b feature/next-task`.

Who works on what

To avoid conflicts, **split by page**, not by file. One person owns one page at a time:

Page	Files they touch
Home	`index.html`, `css/pages/home.css`, `js/pages/home.js`
Flights	`pages/flights.html`, `css/pages/flights.css`, `js/pages/flights.js`
Hotels	`pages/hotels.html`, `css/pages/hotels.css`, `js/pages/hotels.js`
My Trips	`pages/my-trips.html`, `css/pages/my-trips.css`, `js/pages/my-trips.js`
Profile	`pages/profile.html`, `css/pages/profile.css`, `js/pages/profile.js`
Auth	`pages/auth/*.html`, `css/pages/auth.css`, `js/pages/auth.js`
Contact	`pages/contact.html`, `css/pages/contact.css`, `js/pages/contact.js`

> **Shared files** (`css/base.css`, `css/components.css`, `js/main.js`) — coordinate in the group chat **before** editing. These cause the most merge conflicts.

When something goes wrong

"I have unsaved changes but need to pull"

```
git stash          # tuck them away
git pull
git stash pop      # bring them back
```

"I committed to the wrong branch"

```
git log -1                    # copy the commit hash
git reset --soft HEAD~1       # undo the commit, keep changes staged
git checkout correct-branch
git commit -m "..."
```

"Merge conflict"

1. Open the file — find the `<<<<<<<` markers

2. Decide which version to keep (or combine both)
3. Delete the markers
4. `git add <file>` then `git commit`
5. If lost, ask the group chat **before** running anything destructive

Never do these without asking the team

> - `git push --force` — rewrites history for everyone
> - `git reset --hard` on a shared branch
> - Pushing directly to `main` — always go through a PR
> - Committing secrets (`.env` , passwords, API keys)

Karim's checklist (right now)

1. Create the GitHub repo at `github.com/new` — name it `buzz-fly` , do not add README/.gitignore
2. Run the 3 commands GitHub shows you (`git remote add` , `git branch -M main` , `git push -u origin main`) — these connect your **already-existing** local repo to the empty GitHub repo
3. Add the 5 teammates as collaborators (Settings → Collaborators)
4. Send them the repo URL — each one runs `git clone <url>` once
5. From then on, everyone (including you) follows the daily workflow above